

IMPERIAL COLLEGE LONDON

DEPARTMENT OF AERONAUTICS

Numerical Ramjet Engine

Author:

KANISHKA AWASTHI (CID: 06058930)

SHIH-HSUAN TUAN (CID: 06044321)

YIBO ZHU (CID: 06049858)

PRAISE NWACHUKWU (CID: 06019217)

Date: March 12, 2026

Contents

1	Introduction	4
2	Methods	5
2.1	Parameters	5
2.2	Assumptions and Equations	6
2.2.1	Assumptions	6
2.2.2	General Equations	7
2.3	Diffuser	8
2.3.1	Shock Mach Number (M_s)	8
2.3.2	Two-Case Shock Logic	8
2.3.3	Burner-Entry Conditions and Area Sizing	8
2.4	Combustion	9
2.4.1	Inputs conditions	9
2.4.2	Mass Conservation to obtain the mass flow rate	9
2.4.3	Momentum conservation to find the Mb	9
2.5	Nozzle Flow Model	9
2.5.1	Choked Flow Condition	10
2.5.2	Exit Mach Number	10
2.5.3	Nozzle Expansion Ratio	10
2.6	Source Code	11
3	Results	29
3.1	Verification and Validation	29
3.1.1	Inputs	29
3.1.2	Outputs	29
3.2	Plots and Justification	30
4	Conclusion	33
4.1	Extended Critical Analysis:	33

List of Tables

1	Station Definition	5
2	Lists of Inputs Required	29
3	Lists of Results between Codes and Reference	29

List of Figures

1	Configuration of Ramjet Engine	4
2	Efficiency vs. Free Stream Pressure	30
3	Efficiency vs. Free Stream Temperature	30
4	Efficiency vs. Flight Mach Number	30

5	Efficiency vs. Normal Shock Strength	30
6	Efficiency vs. Burner entry mach number	31
7	Efficiency vs. Burner temperature	31
8	Efficiency vs. Burner Pressure Ratio	32
9	Efficiency vs. Required Thrust	32
10	Efficiency vs. Exhaust Pressure Ratio	32

1 Introduction

Ramjet engines have been considered the simplest type of jet engines, making them suitable for high-speed operation (Kumar & Yadav, 2022). The fundamental operating principle of a ramjet engine involves three sequential processes: air intake and control of normal shock in the inlet diffuser, fuel injection and combustion in the combustion chamber, and thrust generation through the exhaust nozzle, as shown in Figure 1.

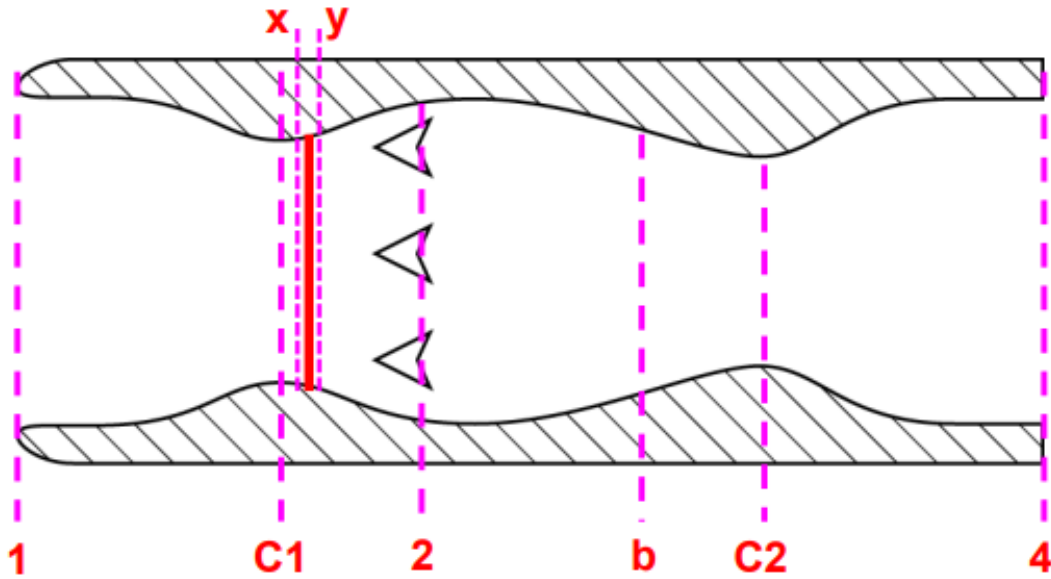


Figure 1: Configuration of Ramjet Engine

Various methods for analyzing performance have been investigated including the gradual variation of input parameters and the replacement of components. (Kumar & Yadav, 2022) (Wéber & Luidens, 1956). This report presents the conceptual sizing and configuration of ramjet engines, along with the performances throughout the consecutive processes — namely, inlet diffusion, fuel combustion, and nozzle exhaustion, using theoretically grounded numerical methods. All individual parameters employed in the subsequent analysis are defined in Section 2.1. The governing assumptions and general equations are established in Section 2.2 to reduce the complexity of the overall calculations. Section 2.3 through Section 2.5 introduce explorations of the characteristic processes of ramjet engines, while the corresponding source codes are included in Section 2.6 to document the complete analytical procedures.

Throughout the analysis, reference data provided in lecture materials (Birch, 2026) are applied to verify the accuracy of results produced by the source codes, as demonstrated Section 3.1. The performance characteristics of ramjet engines under various operating conditions are then illustrated through graphical figures accompanied by critical discussion in Section 3.2.

2 Methods

2.1 Parameters

According to Figure 1, all station parameters are listed in Table 1. The following parameters are indexed according to their respective station numbers.

Table 1: Station Definition

Station	Definition
1	Inlet/Freestream Conditions
C1	Inlet Throat
x	Immediately Before Normal Shock
y	Immediately After Normal Shock
2	Burner Entry
b	Burning Zone/Exit
C2	Nozzle Throat
4	Nozzle Exit

Geometric Parameters

A_i Area at Station i [m^2]

A^* Sonic throat area [m^2]

Thermodynamic Parameters

T_i Static temperature at station i [K]

γ Ratio of specific heats [–]

T_{0i} Stagnation temperature at station i [K]

c_p Specific heat at constant pressure [$\text{J kg}^{-1} \text{K}^{-1}$]

P_i Static pressure at station i [Pa]

R Specific gas constant [$\text{J kg}^{-1} \text{K}^{-1}$]

P_{0i} Stagnation pressure at station i [Pa]

h Specific enthalpy [J kg^{-1}]

ρ_i Density at station i [kg m^{-3}]

Q_f Fuel heating value [J kg^{-1}]

Flow and Velocity Parameters

M_i Mach number at station i [–]

$\dot{m}$ Total mass flow rate [kg s^{-1}]

u_i Flow velocity at station i [m s^{-1}]

$\dot{m}_f$ Fuel mass flow rate [kg s^{-1}]

a_i Speed of sound at station i [m s^{-1}]

$\dot{m}_a$ Air mass flow rate [kg s^{-1}]

Engine Performance Parameters

F Net thrust [N]	η_p Propulsive efficiency [–]
η_{th} Thermal efficiency [–]	η_o Overall efficiency [–]

2.2 Assumptions and Equations

2.2.1 Assumptions

Several assumptions were made to simplify the analysis and allow the quasi-one-dimensional calculations to be performed. These are outlined below.

1. **Constant ratio of specific heat capacities.** The ratio of specific heats, γ , is assumed to remain constant throughout the engine at $\gamma = 1.4$, consistent with calorically perfect air.
2. **Isentropic flow everywhere except across shocks and Combustor.** All compressions and expansions are assumed isentropic. With the exception of the normal shock at the diffuser inlet (and oblique shock in alternative inlet configurations), entropy increases and stagnation pressure losses occur during this process. Additionally, heat input occurs during combustion, resulting in entropy increase.
3. **Adiabatic flow everywhere except the combustor.** No heat transfer occurs between the flow and the engine walls in the diffuser and nozzle. Combustor is an ideal heat addition model. Consequently, the stagnation temperature T_0 is conserved through these modules.
4. **Negligible fuel-to-air ratio.** The mass flow rate of fuel is assumed small relative to the air mass flow rate, such that $f = \dot{m}_f/\dot{m}_a \ll 1$. The total mass flow rate $\dot{m}$ is therefore treated as constant throughout the engine.
5. **Exhaust gases have the same thermodynamic properties as air.** The combustion products are assumed to have the same specific gas constant $R = 287 \text{ J/kg}\cdot\text{K}$ and ratio of specific heats $\gamma = 1.4$ as the incoming air.
6. **Instantaneous heat addition in the burner.** All heat is assumed to be added instantaneously and uniformly across the burner cross-section.
7. **Stagnation properties conserved through isentropic processes.** As a direct consequence of assumptions 2 and 3, stagnation pressure P_0 and stagnation temperature T_0 remain constant through the nozzle, permitting the use of isentropic relations between suitable stations except for shock location.
8. **Constant mass flow rate throughout the ramjet duct.** The duct assumes a non-varying mass flow rate that is derived from conservation of momentum - thrust per mass flow equation as pointed out in the lectures.
9. **No occurrence of complex phenomenas.** No complex phenomenas such as shock-wave boundary layer interactions, oblique shock series accumulation, etc. considered in engine study at any point.

2.2.2 General Equations

Isentropic Relations

$$\frac{T_0}{T} = 1 + \frac{\gamma - 1}{2} M^2 \quad (1)$$

$$\frac{P_0}{P} = \left[1 + \frac{\gamma - 1}{2} M^2 \right]^{\frac{\gamma}{\gamma - 1}} \quad (2)$$

$$\frac{A}{A^*} = \frac{1}{M} \left[\frac{2}{\gamma + 1} \left(1 + \frac{\gamma - 1}{2} M^2 \right) \right]^{\frac{\gamma + 1}{2(\gamma - 1)}} \quad (3)$$

Normal Shock Relation

$$M_2 = \sqrt{\frac{(\gamma - 1)M_1^2 + 2}{2\gamma M_1^2 - (\gamma - 1)}} \quad (4)$$

$$\frac{T_2}{T_1} = \frac{[(\gamma - 1)M_1^2 + 2][2\gamma M_1^2 - (\gamma - 1)]}{(\gamma + 1)^2 M_1^2} \quad (5)$$

$$\frac{P_2}{P_1} = \frac{[2\gamma M_1^2 - (\gamma - 1)]}{(\gamma + 1)} \quad (6)$$

Performance Equation

$$\frac{F}{P_1 A_1} = \gamma M_1^2 \left[\frac{M_4^2 A_4}{M_1^2 A_1} - 1 \right] \quad (7)$$

Propulsive Efficiency Equation

$$\eta_p = \frac{F V_1}{\frac{1}{2} \dot{m} (V_4^2 - V_1^2)} \quad (8)$$

Thermodynamic Efficiency Equation

$$\eta_{th} = \frac{\frac{1}{2} \dot{m} (V_4^2 - V_1^2)}{\dot{m} c_p (T_{0b} - T_{02})} \quad (9)$$

Total Efficiency Equation

$$\eta_0 = \eta_T \eta_P \quad (10)$$

2.3 Diffuser

2.3.1 Shock Mach Number (M_s)

The normal shock strength is specified directly as the static pressure ratio P_{C1}/P_1 (input c). The upstream shock Mach number M_s is derived by inverting the Rankine-Hugoniot pressure relation:

$$M_s = \sqrt{\frac{\frac{P_{C1}}{P_1}(\gamma + 1) + (\gamma - 1)}{2\gamma}} \quad (11)$$

Physical validity requires $1 < M_s \leq M_1$, thus for $M_s > M_1$ the shock cannot be sustained inside the inlet, and engine unstart occurs.

2.3.2 Two-Case Shock Logic

The diffuser model distinguishes two cases based on the magnitude M_s relative to the critical threshold of 1.3:

- **Case 1** ($M_s \geq 1.3$): The flow first chokes at the inlet throat C_1 ($M = 1$), after which the diverging duct re-accelerates the flow isentropically back to M_s before the normal shock occurs. Stagnation conditions upstream of the shock are therefore $P_{0,\text{pre}} = P_{01}$ and $T_{0,\text{pre}} = T_{01}$.
- **Case 2** ($M_s < 1.3$): The shock is sufficiently weak that it occurs directly from freestream conditions without prior choking, so $P_{0,\text{pre}} = P_{01}$ static conditions upstream of the shock are those of the freestream.

In both cases, the stagnation pressure loss is computed via the Rayleigh-Pitot formula applied at M_s , and stagnation temperature is unchanged across the shock (adiabatic condition):

$$\frac{P_{02}}{P_{01}} = \left(\frac{(\gamma + 1)M_s^2}{2 + (\gamma - 1)M_s^2} \right)^{\gamma/(\gamma-1)} \left(\frac{\gamma + 1}{2\gamma M_s^2 - (\gamma - 1)} \right)^{1/(\gamma-1)}, \quad T_{02} = T_{01} \quad (12)$$

2.3.3 Burner-Entry Conditions and Area Sizing

Post-shock, the flow diffuses isentropically to the prescribed burner-entry Mach number M_2 , conserving P_{02} and T_{02} . Static conditions at Station 2 follow directly from the isentropic relations (Eqs. 1 and 2). All station areas are obtained from mass continuity:

$$A_i = \frac{\dot{m}}{\rho_i u_i}, \quad A_{C1} = \frac{A_1}{A/A^*|_{M_1}} \quad (13)$$

where the mass flow rate $\dot{m}$ is scaled from the required thrust F , and $A/A^*|_{M_1}$ is the isentropic area-Mach ratio evaluated at the freestream Mach number M_1 .

2.4 Combustion

The combustor increases the flow temperature through heat addition while maintaining subsonic flow conditions. The burner entry conditions (P_2, T_2, M_2, A_2) are obtained from the diffuser model.

2.4.1 Inputs conditions

Two parameters are prescribed for the combustor:

- Burner exit static temperature T_b
- Burner pressure ratio P_b/P_2

2.4.2 Mass Conservation to obtain the mass flow rate

Assuming that the fuel mass flow is negligible compared with the air mass flow, the mass flow rate remains constant across the combustor:

$$\dot{m} = \rho_2 a_2 M_2 A_2 = \rho_b a_b M_b A_b \quad (14)$$

2.4.3 Momentum conservation to find the M_b

The burner exit Mach number M_b is obtained by applying the steady one-dimensional momentum equation to the control volume:

$$(\rho_2 u_2^2 + P_2) A_2 = (\rho_b u_b^2 + P_b) A_b \quad (15)$$

Substituting these relations into the momentum equation results in a nonlinear equation for the unknown exit Mach number M_b . This equation is solved numerically within the subsonic range

$$0 < M_b < 1 \quad (16)$$

A root-finding method is used to determine the physically valid solution.

2.5 Nozzle Flow Model

The nozzle accelerates the combustion gases by converting thermal energy into kinetic energy through an isentropic expansion. The flow entering the nozzle is characterised by the stagnation pressure P_{0b} and stagnation temperature T_{0b} at the combustor exit.

2.5.1 Choked Flow Condition

For compressible flow through a converging–diverging nozzle, choking occurs when the pressure ratio across the nozzle falls below the critical value

$$\frac{P_e}{P_{0b}} < \left(\frac{P_e}{P_{0b}} \right)_{crit} = \left(\frac{2}{\gamma + 1} \right)^{\frac{\gamma}{\gamma-1}} \quad (17)$$

since the flow becomes choked and the Mach number at the nozzle throat is $M^* = 1$. Under choked conditions the mass flow rate through the nozzle becomes fixed by the throat area and the stagnation conditions. Using isentropic relations, the static temperature at the throat is:

$$T^* = \frac{T_{0b}}{1 + \frac{\gamma-1}{2} M^{*2}}, \quad \text{and since } M^* = 1, \quad T^* = \frac{T_{0b}}{1 + \frac{\gamma-1}{2}} \quad (18)$$

The flow static pressure, ideal density, speed of sound, and velocity at the throat are given by Eqs. (23) and (13)–(15), with the notation adapted for the nozzle geometry shown in Fig. 1. Mass conservation (continuity) determines the required throat area A^* . Thus, the nozzle throat area is fixed by the required engine mass flow rate and the stagnation conditions.

2.5.2 Exit Mach Number

The exit Mach number is obtained from the isentropic pressure relation.

$$\frac{P_e}{P_{0b}} = \left(1 + \frac{\gamma-1}{2} M_e^2 \right)^{-\frac{\gamma}{\gamma-1}}. \quad (19)$$

Once M_e is known, the exit temperature, pressure, density, speed of sound, and velocity follow from the isentropic relations using standard thermodynamic tables or the equations:

$$T_e = \frac{T_{0b}}{1 + \frac{\gamma-1}{2} M_e^2}, \quad P_e = \frac{P_{0b}}{\left(1 + \frac{\gamma-1}{2} M_e^2 \right)^{\frac{\gamma}{\gamma-1}}}, \quad \rho_e = \frac{P_e}{RT_e}, \quad a_e = \sqrt{\gamma RT_e}, \quad u_e = M_e a_e.$$

2.5.3 Nozzle Expansion Ratio

The required exit area follows from the area–Mach relation

$$\frac{A_e}{A^*} = \frac{1}{M_e} \left[\frac{2}{\gamma+1} \left(1 + \frac{\gamma-1}{2} M_e^2 \right) \right]^{\frac{\gamma+1}{2(\gamma-1)}}, \quad (20)$$

so that

$$A_e = A^* \left(\frac{A_e}{A^*} \right). \quad (21)$$

2.6 Source Code

Listing 1: MATLAB Ramjet code

```

1 clear; clc; close all;
2
3 % GAS PROPERTIES
4 gamma = 1.4; % Ratio of specific heats
5 R      = 287; % Gas constant for air (J/kg/K)
6
7 % BASELINE INPUTS (coded as in the figure)
8 P1      = 70e3; % (a) Free-stream pressure at station 1
9 T1      = 210; % (a) Free-stream temperature at station 1
10 M1     = 2.8; % (b) Flight Mach number at station 1
11 PC1_P1 = 1.245; % (c) Normal shock strength P2/P1
12 M2     = 0.2; % (d) Burner entry Mach number at station 2
13 Tb     = 1700; % (e) Burner temperature at station b
14 Pb_P2  = 1.0; % (f) Burner pressure ratio Pb/P2
15 P4_P1  = 1.0; % (g) Exhaust pressure ratio P4/P1
16 F_req  = 2000; % (h) Required thrust
17
18 Pe = P4_P1 * P1; % Exhaust static pressure at station 4
19
20 % Example Data calculation
21 [eta_th, eta_p, result] = single_point( ...
22     P1, T1, M1, PC1_P1, M2, F_req, Tb, Pb_P2, Pe, gamma, R);
23 % Solve the Lecture 3 example data to validation check
24
25 % Print the required outputs
26 fprintf(' REQUIRED OUTPUTS\n');
27 fprintf('(a) Inlet area      A1 = %.6f m^2\n', result.A1);
28 fprintf('(b) Inlet throat area AC1 = %.6f m^2\n', result.AC1);
29 fprintf('(c) Burner entry area A2 = %.6f m^2\n', result.A2);
30 fprintf('(d) Burner exit area Ab = %.6f m^2\n', result.Ab);
31 fprintf('(e) Nozzle throat area AC2 = %.6f m^2\n', result.AC2);
32 fprintf('(f) Exhaust area      A4 = %.6f m^2\n', result.A4);
33 fprintf('(g) Thermodynamic eff. eta_th = %.6f\n', eta_th);
34 fprintf('(h) Propulsive eff.    eta_p = %.6f\n', eta_p);
35
36 % SWEEP settings to change the input parameters (vectored range
    of values for plotting purposes)
37 N = 120; % Number of points used in each parameter sweep
38 % 1) FREE-STREAM PRESSURE P1
39 P1_vec = linspace(0, 100e3, N);
40 % Sweep range for free-stream pressure
41 [eta_th_P1, eta_p_P1, eta_o_P1, valid_P1, status_P1] = ...
42     sweep_P1(P1_vec, T1, M1, PC1_P1, M2, F_req, Tb, Pb_P2,
        P4_P1, gamma, R); % Sweep P1

```

```

43 plot_efficiencies_choking(P1_vec ./ 1000, eta_th_P1, eta_p_P1,
    eta_o_P1, valid_P1, ...
44     'Free-stream Pressure P_1 (kPa)', ...
45     'Efficiency vs Free-stream Pressure');
46     % Plot efficiencies for P1 sweep
47 print_status_summary('P1 sweep', status_P1);
48 % Print feasibility summary for P1 sweep
49
50 % 2) FREE-STREAM TEMPERATURE T1
51 T1_vec = linspace(0, 1000, N);
52 % Sweep range for free-stream temperature
53 [eta_th_T1, eta_p_T1, eta_o_T1, valid_T1, status_T1] = ...
54     sweep_T1(T1_vec, P1, M1, PC1_P1, M2, F_req, Tb, Pb_P2,
    P4_P1, gamma, R);
55 % Sweep T1
56 plot_efficiencies_choking(T1_vec, eta_th_T1, eta_p_T1, eta_o_T1
    , valid_T1, ...
57     'Free-stream Temperature T_1 (K)', ...
58     'Efficiency vs Free-stream Temperature');
59 % Plot efficiencies for T1 sweep
60 print_status_summary('T1 sweep', status_T1);
61 % Print feasibility summary for T1 sweep
62
63 % 3) FLIGHT MACH NUMBER M1
64 M1_vec = linspace(0, 10.0, N);
65 % Sweep range for flight Mach number
66 [eta_th_M1, eta_p_M1, eta_o_M1, valid_M1, status_M1] = ...
67     sweep_M1(M1_vec, P1, T1, PC1_P1, M2, F_req, Tb, Pb_P2,
    P4_P1, gamma, R);
68 % Sweep M1
69 plot_efficiencies_choking(M1_vec, eta_th_M1, eta_p_M1, eta_o_M1
    , valid_M1, ...
70     'Flight Mach Number M_1', ...
71     'Efficiency vs Flight Mach Number');
72 % Plot efficiencies for M1 sweep
73 print_status_summary('M1 sweep', status_M1);
74 % Print feasibility summary for M1 sweep
75
76 % 4) NORMAL SHOCK STRENGTH P2/P1
77 sp_vec = linspace(1.0, 7.0, N);
78 % Sweep range for normal shock strength
79 [eta_th_sp, eta_p_sp, eta_o_sp, valid_sp, status_sp] = ...
80     sweep_sp(sp_vec, P1, T1, M1, M2, F_req, Tb, Pb_P2, P4_P1,
    gamma, R);
81 % Sweep shock strength
82 plot_efficiencies_choking(sp_vec, eta_th_sp, eta_p_sp, eta_o_sp
    , valid_sp, ...
83     'Normal Shock Strength P_2/P_1', ...

```

```

84     'Efficiency vs Normal Shock Strength');
85 % Plot efficiencies for shock sweep
86 print_status_summary('Shock sweep', status_sp);
87 % Print feasibility summary for shock sweep
88
89 % 5) BURNER ENTRY MACH NUMBER M2
90 M2_vec = linspace(0.10, 0.50, N);
91 % Sweep range for burner entry Mach number
92 [eta_th_M2, eta_p_M2, eta_o_M2, valid_M2, status_M2] = ...
93     sweep_M2(M2_vec, P1, T1, M1, PC1_P1, F_req, Tb, Pb_P2,
94         P4_P1, gamma, R);
95 % Sweep M2
96 plot_efficiencies_choking(M2_vec, eta_th_M2, eta_p_M2, eta_o_M2
97     , valid_M2, ...
98     'Burner Entry Mach Number M_2', ...
99     'Efficiency vs Burner Entry Mach Number');
100 % Plot efficiencies for M2 sweep
101 print_status_summary('M2 sweep', status_M2);
102 % Print feasibility summary for M2 sweep
103
104 % 6) BURNER TEMPERATURE Tb
105 Tb_vec = linspace(1200, 2200, N);
106 % Sweep range for burner exit temperature
107 [eta_th_Tb, eta_p_Tb, eta_o_Tb, valid_Tb, status_Tb] = ...
108     sweep_Tb(Tb_vec, P1, T1, M1, PC1_P1, M2, F_req, Pb_P2,
109         P4_P1, gamma, R);
110 % Sweep Tb
111 plot_efficiencies_choking(Tb_vec, eta_th_Tb, eta_p_Tb, eta_o_Tb
112     , valid_Tb, ...
113     'Burner Temperature T_b (K)', ...
114     'Efficiency vs Burner Temperature');
115 % Plot efficiencies for Tb sweep
116 print_status_summary('Tb sweep', status_Tb);
117 % Print feasibility summary for Tb sweep
118
119 % 7) BURNER PRESSURE RATIO Pb/P2
120 PbP2_vec = linspace(0.90, 1.00, N);
121 % Sweep range for burner pressure ratio
122 [eta_th_PbP2, eta_p_PbP2, eta_o_PbP2, valid_PbP2, status_PbP2]
123     = ...
124     sweep_PbP2(PbP2_vec, P1, T1, M1, PC1_P1, M2, F_req, Tb,
125         P4_P1, gamma, R);
126 % Sweep Pb/P2
127 plot_efficiencies_choking(PbP2_vec, eta_th_PbP2, eta_p_PbP2,
128     eta_o_PbP2, valid_PbP2, ...
129     'Burner Pressure Ratio P_b/P_2', ...
130     'Efficiency vs Burner Pressure Ratio');
131 % Plot efficiencies for Pb/P2 sweep

```

```

125 print_status_summary('Pb/P2 sweep', status_PbP2);
126 % Print feasibility summary for Pb/P2 sweep
127
128 % 8) REQUIRED THRUST F
129 F_vec = linspace(500, 10000, N);
130 % Sweep range for required thrust
131 [eta_th_F, eta_p_F, eta_o_F, valid_F, status_F] = ...
132     sweep_F(F_vec, P1, T1, M1, PC1_P1, M2, Tb, Pb_P2, P4_P1,
133         gamma, R);
134 % Sweep thrust
135 plot_efficiencies_choking(F_vec, eta_th_F, eta_p_F, eta_o_F,
136     valid_F, ...
137     'Required Thrust F (N)', ...
138     'Efficiency vs Required Thrust');
139 % Plot efficiencies for thrust sweep
140 print_status_summary('Thrust sweep', status_F);
141 % Print feasibility summary for thrust sweep
142
143 % 9) EXHAUST PRESSURE RATIO P4/P1
144 P4P1_vec = linspace(0.2, 1.0, N);
145 % Sweep range for exhaust pressure ratio
146 [eta_th_P4P1, eta_p_P4P1, eta_o_P4P1, valid_P4P1, status_P4P1]
147     = ...
148     sweep_P4P1(P4P1_vec, P1, T1, M1, PC1_P1, M2, F_req, Tb,
149         Pb_P2, gamma, R);
150 % Sweep P4/P1
151 plot_efficiencies_choking(P4P1_vec, eta_th_P4P1, eta_p_P4P1,
152     eta_o_P4P1, valid_P4P1, ...
153     'Exhaust Pressure Ratio P_4/P_1', ...
154     'Efficiency vs Exhaust Pressure Ratio');
155 % Plot efficiencies for P4/P1 sweep
156 print_status_summary('P4/P1 sweep', status_P4P1);
157 % Print feasibility summary for P4/P1 sweep
158
159 % SINGLE-POINT SOLVER: Validates the Lecture 3 example data.
160 % Contains 3 single flow solvers: diffuser, combustion and
161 % nozzle, to compute the required output parameters for each
162 % station.
163 function [eta_th, eta_p, out] = single_point(P1, T1, M1, sp, M2
164     , F_req, Tb, PbP2, Pe, gamma, R)
165
166 % The unit mass flow below is used to determine the specific
167 % thrust which is in turn used to evaluate the overall mass
168 % flow rate (considered constant throughout the ramjet duct)
169 % using the Thrust per unit mass flow equation (from
170 % conservation of momentum).
171 m_dot_unit = 1.0; % Unit mass flow rate (kg/s)

```

```

160 [P2_u, T2_u, A2_u, mdot2_u, P02_u, T02_u, A1_u, AC1_u, V1] =
    ...
161     Diffuser_SingleFlow(P1, T1, M1, sp, M2, m_dot_unit, gamma,
        R);
162 % Solve diffuser with Unit mass flow
163
164 [Pb_u, Tb_u, Mb_u, Ab_u, mdotb_u, T0b_u, P0b_u, flag_b,
    reason_b] = ...
165     Combustion_SingleFlow(P2_u, T2_u, M2, A2_u, mdot2_u, Tb,
        PbP2, gamma, R);
166 % Solve combustor with unit mass flow
167 stb.Pb = Pb_u; % Save burner-exit static pressure
168 stb.Tb = Tb_u; % Save burner-exit static temperature
169 stb.Mb = Mb_u; % Save burner-exit Mach number
170 stb.Ab = Ab_u; % Save burner-exit area
171 stb.mdot = mdotb_u; % Save burner-exit mass flow rate
172 stb.T0b = T0b_u; % Save burner-exit stagnation temperature
173 stb.P0b = P0b_u; % Save burner-exit stagnation pressure
174 st4_u = Nozzle_SingleFlow(stb, Pe, gamma, R);
175 % Solve nozzle from burner exit to exhaust
176
177 F_per_mdot = (st4_u.u4 - V1) + (st4_u.P4 - Pe) * st4_u.A4 / stb
    .mdot;
178 % Specific thrust (N per kg/s)
179
180 m_dot = F_req / F_per_mdot;
181 %Scale mass flow to match required thrust
182
183 [P2, T2, A2, mdot2, P02, T02, A1, AC1, ~] = ...
184     Diffuser_SingleFlow(P1, T1, M1, sp, M2, m_dot, gamma, R);
185 % Recalculate diffuser with scaled mass flow
186
187 [Pb, Tb, Mb, Ab, mdotb, T0b, P0b, flag_b, reason_b] = ...
188     Combustion_SingleFlow(P2, T2, M2, A2, mdot2, Tb, PbP2,
        gamma, R);
189 % Recalculate combustor with scaled mass flow
190
191 stb.Pb = Pb; % Save final burner-exit static pressure
192 stb.Tb = Tb; % Save final burner-exit static temperature
193 stb.Mb = Mb; % Save final burner-exit Mach number
194 stb.Ab = Ab; % Save final burner-exit area
195 stb.mdot = mdotb; % Save final burner-exit mass flow rate
196 stb.T0b = T0b; % Save final burner-exit stagnation temperature
197 stb.P0b = P0b; % Save final burner-exit stagnation pressure
198
199 st4 = Nozzle_SingleFlow(stb, Pe, gamma, R);
200 % Solve final nozzle state
201

```

```

202 cp = gamma * R / (gamma - 1);
203 % Constant-pressure specific heat
204 F_actual = mdotb * (st4.u4 - V1) + (st4.P4 - Pe) * st4.A4;
205 % Actual thrust
206 jet_power = 0.5 * mdotb * (st4.u4^2 - V1^2);
207 % Useful jet power
208 q_in = mdotb * cp * (T0b - T02);
209 % Heat added in combustor
210
211 eta_th = jet_power / q_in; % Thermodynamic efficiency
212 eta_p = (F_actual * V1) / jet_power; % Propulsive efficiency
213 out.A1 = A1; % Output inlet area at station 1
214 out.AC1 = AC1; % Output inlet throat area at station C1
215 out.A2 = A2; % Output burner entry area at station 2
216 out.Ab = Ab; % Output burner exit area at station b
217 out.AC2 = st4.AC2; % Output nozzle throat area at station C2
218 out.A4 = st4.A4; % Output exhaust area at station 4
219
220 end
221
222 %% DIFFUSER MODULE: 1 -> C1 -> 2
223 %% The diffuser calculates the burner entry parameters from the
    required inputs at the inlet of the ramjet by considering a
    shock position at or after the throat region for a
    reasonable engine self start. It assumes isentropic flow
    before and after the shock, and a constant mass flow rate
    through it. Also no oblique shock series modelling is
    considered nor any shock-wave boundary-layer interactions
    occurring in the diffuser.
224 function [P2, T2, A2, m_dot, P02, T02, A1, AC1, V1] =
    Diffuser_SingleFlow(P1, T1, M1, p2_over_p1, M2, m_dot, gamma
    , R)
225
226 P01 = P1 * (1 + ((gamma - 1) / 2) * M1^2)^(gamma / (gamma - 1))
    ;
227 % Stagnation pressure at station 1
228 T01 = T1 * (1 + ((gamma - 1) / 2) * M1^2);
229 % Stagnation temperature at station 1
230
231 V1 = M1 * sqrt(gamma * R * T1); % Velocity at station 1
232 rho1 = P1 / (R * T1); % Density at station 1
233
234 A1 = m_dot / (rho1 * V1); % Area at station 1
235 AC1 = A1 / area_mach_func(M1, gamma);
236 % Choked throat area at station C1
237
238 if ~isreal(A1) || ~isfinite(A1) || A1 <= 0
239 % Check whether inlet area is physical

```



```

240     error('Diffuser failed: invalid inlet area A1.');
```

```

241 end
242
243 if ~isreal(AC1) || ~isfinite(AC1) || AC1 <= 0
244     % Check whether throat area is physical
245     error('Diffuser failed: invalid inlet throat area AC1.');
```

```

246 end
247
248 Ms = inverse_normal_shock(p2_over_p1, gamma);
249 % Upstream Mach number corresponding to shock strength
250
251 if ~isreal(Ms) || ~isfinite(Ms)
252     % Check whether shock Mach number is real and finite
253     error('Diffuser failed: invalid shock upstream Mach number.
254           ');
255 end
256
257 if Ms <= 1 % Check if shock is physically possible
258     error('Diffuser failed: shock upstream Mach must be greater
259           than 1.');
```

```

260 end
261
262 if M2 <= 0 || M2 >= 1 % Check burner entry Mach number
263     error('Diffuser failed: M2 must satisfy 0 < M2 < 1.');
```

```

264 end
265
266 if Ms >= 1.3
267     % Case 1: shock occurs after choking in diffuser
268     P0pre = P01; % Stagnation pressure before normal shock
269     T0pre = T01; % Stagnation temperature before normal shock
270     Mpre = 1.0; % Flow becomes sonic at station C1
271     Ppre = P0pre / isentropic_press_ratio(Mpre, gamma);
272     % Static pressure before shock
273     Tpre = T0pre / isentropic_temp_ratio(Mpre, gamma);
274     % Static temperature before shock
275     P0post = P0pre * norm_shock_press_stag(Ms, gamma);
276     % Stagnation pressure after shock
277     T0post = T0pre;
278     % Stagnation temperature unchanged across shock
279
280 else % Case 2: shock occurs directly from freestream
281
282     Ppre = P1; % Shock starts from freestream static pressure
283     Tpre = T1; % Shock starts from freestream static
284         temperature

```

```

285     P0post = P01 * norm_shock_press_stag(Ms, gamma);
286 % Stagnation pressure after shock
287     T0post = T01; % Stagnation temperature after shock
288 end
289
290 P02 = P0post; % Stagnation pressure at station 2
291 T02 = T0post; % Stagnation temperature at station 2
292
293 P2 = P02 / isentropic_press_ratio(M2, gamma);
294 % Static pressure at station 2
295 T2 = T02 / isentropic_temp_ratio(M2, gamma);
296 % Static temperature at station 2
297
298 if ~isreal(T2) || ~isfinite(T2) || T2 <= 0
299 % Check whether station-2 temperature is physical
300     error('Diffuser failed: invalid station-2 temperature.');
```

301 end

302

```

303 rho2 = P2 / (R * T2); % Density at station 2
304 V2 = M2 * sqrt(gamma * R * T2); % Velocity at station 2
305 A2 = m_dot / (rho2 * V2); % Area at station 2
306
307 if ~isreal(A2) || ~isfinite(A2) || A2 <= 0
308 % Check whether burner-entry area is physical
309     error('Diffuser failed: invalid burner entry area A2.');
```

310 end

311 end

312

```

313 %% COMBUSTOR MODULE: 2 -> b
314 %% The combustor takes the output of the diffuser as the input
    for the burner entry region at station 2 and computes the
    various parameter values at station 'b' considering an ideal
    heat addition model. It neglects the fuel mass
    considerations (so m_dot nearly remains constant) and drag
    considerations in the combustion chamber.
```

```

315 function [Pb, Tb, Mb, Ab, mdotb, T0b, P0b, flag, reason] = ...
316     Combustion_SingleFlow(P2, T2, M2, A2, mdot2, Tb_in, Pb_P2,
    gamma, R)
317
318 Pb = NaN; Tb = NaN; Mb = NaN; Ab = NaN;
319 % Initialise burner-exit variables
320 mdotb = NaN; T0b = NaN; P0b = NaN;
321 % Initialise burner-exit total properties
322 flag = 0; reason = "OK"; % Default success flag
323
324 if any([Tb_in, Pb_P2] <= 0) % Check combustor inputs
325     flag = 3; reason = "Invalid combustor inputs."; return;
326 % Exit if invalid
```

```

327 end
328
329 if M2 >= 1 % Require subsonic combustor inlet
330     flag = 3; reason = "M2 must be subsonic."; return;
331 % Exit if invalid
332 end
333
334 if Tb_in <= T2 % Require heating through combustor
335     flag = 3; reason = "Tb must be greater than T2."; return;
336 % Exit if invalid
337 end
338
339 Tb = Tb_in; % Burner-exit static temperature
340 Pb = P2 * Pb_P2; % Burner-exit static pressure
341
342 rho2 = P2 / (R * T2); % Density at station 2
343 a2 = sqrt(gamma * R * T2); % Speed of sound at station 2
344 u2 = M2 * a2; % Velocity at station 2
345
346 left_mom = (rho2 * u2^2 + P2) * A2; % Momentum-flux term at
    station 2
347 residual = @(Mb) obtainMb(Mb, left_mom, mdot2, Pb, Tb, gamma, R
    );
348 % Residual for burner-exit Mach number
349
350 Mb_scan = linspace(1e-4, 0.999, 400);
351 % Scan subsonic Mach range
352 f_scan = arrayfun(residual, Mb_scan);
353 % Evaluate residual over scan range
354
355 idx = find(isfinite(f_scan(1:end-1)) & isfinite(f_scan(2:end))
    & ...
    (f_scan(1:end-1) .* f_scan(2:end) < 0), 1);
356 % Find a valid sign change interval
357
358
359 if isempty(idx)
360 % Check if no valid root exists
361     flag = 1; reason = "No feasible subsonic solution for Mb.";
        return; % Exit if no solution
362 end
363
364 Mb = fzero(residual, [Mb_scan(idx), Mb_scan(idx+1)]);
365 % Solve burner-exit Mach number
366
367 if ~isreal(Mb) % Check whether Mb is complex
368     flag = 2; reason = "Complex Mach number at burner exit.";
        return;
369 end

```

```

370
371 if ~isfinite(Mb) % Check whether Mb is finite
372     flag = 2; reason = "Non-finite Mach number at burner exit
        ."; return;
373 end
374
375 if Mb <= 0 || Mb >= 1
376 % Check whether Mb is physically admissible
377     flag = 2; reason = "Nonphysical combustor solution: Mb must
        satisfy 0 < Mb < 1."; return;
378 end
379
380 rhob = Pb / (R * Tb); % Density at station b
381 ab = sqrt(gamma * R * Tb); % Speed of sound at station b
382 ub = Mb * ab; % Velocity at station b
383 Ab = mdot2 / (rhob * ub); % Area at station b
384
385 mdotb = rhob * ub * Ab; % Mass flow rate at station b
386 T0b = Tb * (1 + (gamma - 1) / 2 * Mb^2);
387 % Stagnation temperature (Assume at the section b, it's
        isentropic)
388 P0b = Pb * (1 + (gamma - 1) / 2 * Mb^2)^(gamma / (gamma - 1));
389 % Stagnation pressure at station b
390
391 end
392
393 %% NOZZLE MODULE: b -> C2 -> 4
394 function st4 = Nozzle_SingleFlow(stb, Pe, gamma, R)
395
396 st4 = struct(); % Create output structure for nozzle
397 st4.AC2 = NaN; st4.A4 = NaN; % Initialise required areas
398 st4.P4 = NaN; st4.T4 = NaN; st4.M4 = NaN;
399 % Initialise exhaust state
400 st4.u4 = NaN; % Initialise exhaust velocity
401 st4.flag = 0; st4.reason = "OK"; % Default success flag
402
403 P0b = stb.P0b; % Burner-exit stagnation pressure
404 T0b = stb.T0b; % Burner-exit stagnation temperature
405 mdot = stb.mdot; % Burner-exit mass flow rate
406
407 pcrit = (2 / (gamma + 1))^(gamma / (gamma - 1));
408 % Critical pressure ratio for choking
409
410 if Pe / P0b > pcrit % Check if nozzle is choked
411     st4.flag = 2; % Set error flag
412     st4.reason = "Nozzle not choked; current model assumes
        choking.";
413 % Explain failure

```

```

414     return;
415 end
416
417 MC2 = 1.0; % Nozzle throat Mach number at station C2
418 TC2 = T0b / (1 + (gamma - 1) / 2 * MC2^2);
419 % Static temperature at station C2
420 PC2 = P0b / (1 + (gamma - 1) / 2 * MC2^2)^(gamma / (gamma - 1))
421 ;
422 % Static pressure at station C2
423 rhoC2 = PC2 / (R * TC2); % Density at station C2
424 aC2 = sqrt(gamma * R * TC2); % Speed of sound at station C2
425 uC2 = MC2 * aC2; % Velocity at station C2
426 AC2 = mdot / (rhoC2 * uC2); % Area at station C2
427
428 M4 = solve_exit_mach(Pe / P0b, gamma);
429 % Solve exhaust Mach number at station 4
430
431 if ~isreal(M4)
432 % Check whether exhaust Mach number is complex
433     st4.flag = 2; st4.reason = "Complex exhaust Mach number.";
434     return;
435 end
436
437 if ~isfinite(M4) || M4 <= 1
438 % Check whether exhaust Mach number is physical
439     st4.flag = 2; st4.reason = "Nonphysical exhaust Mach number
440     ."; return;
441 end
442
443 T4 = T0b / (1 + (gamma - 1) / 2 * M4^2);
444 % Static temperature at station 4
445 P4 = P0b / (1 + (gamma - 1) / 2 * M4^2)^(gamma / (gamma - 1));
446 % Static pressure at station 4
447
448 rho4 = P4 / (R * T4); % Density at station 4
449 a4 = sqrt(gamma * R * T4); % Speed of sound at station 4
450 u4 = M4 * a4; % Velocity at station 4
451 A4 = AC2 * area_mach_func(M4, gamma);
452 % Area at station 4
453
454 mdot_check = rho4 * u4 * A4;
455 % Check mass flow consistency
456
457 if abs(mdot_check - mdot) / mdot > 0.01
458 % Check if mass conservation error exceeds 1%
459     st4.flag = 1;
460 % Set error flag

```

```

459     st4.reason = "Mass conservation mismatch.";
460 % Explain failure
461 end
462 st4.AC2 = AC2;    % Save nozzle throat area
463 st4.A4  = A4;    % Save exhaust area
464 st4.P4  = P4;    % Save exhaust static pressure
465 st4.T4  = T4;    % Save exhaust static temperature
466 st4.M4  = M4;    % Save exhaust Mach number
467 st4.u4  = u4;    % Save exhaust velocity
468 end
469
470 % SWEEP FUNCTIONS (for the SWEEP vectored values used in the
    plotting of graphs.)
471 function [eta_th, eta_p, eta_o, valid, status] = sweep_P1(
    P1_vec, T1, M1, sp, M2, F_req, Tb, PbP2, P4P1, gamma, R)
472 N = numel(P1_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
    NaN(1,N); valid = false(1,N); status = zeros(1,N);
473 for k = 1:N
474     try
475         Pe = P4P1 * P1_vec(k);
476         [eth, ep] = single_point(P1_vec(k), T1, M1, sp, M2,
            F_req, Tb, PbP2, Pe, gamma, R);
477         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
478         valid(k) = true; status(k) = 0;
479     catch ME
480         valid(k) = false; status(k) = classify_error(ME.message
            );
481     end
482 end
483 end
484
485 function [eta_th, eta_p, eta_o, valid, status] = sweep_T1(
    T1_vec, P1, M1, sp, M2, F_req, Tb, PbP2, P4P1, gamma, R)
486 N = numel(T1_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
    NaN(1,N); valid = false(1,N); status = zeros(1,N);
487 for k = 1:N
488     try
489         Pe = P4P1 * P1;
490         [eth, ep] = single_point(P1, T1_vec(k), M1, sp, M2,
            F_req, Tb, PbP2, Pe, gamma, R);
491         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
492         valid(k) = true; status(k) = 0;
493     catch ME
494         valid(k) = false; status(k) = classify_error(ME.message
            );
495     end
496 end
497 end

```

```

498
499 function [eta_th, eta_p, eta_o, valid, status] = sweep_M1(
      M1_vec, P1, T1, sp, M2, F_req, Tb, PbP2, P4P1, gamma, R)
500 N = numel(M1_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
      NaN(1,N); valid = false(1,N); status = zeros(1,N);
501 for k = 1:N
502     try
503         Pe = P4P1 * P1;
504         [eth, ep] = single_point(P1, T1, M1_vec(k), sp, M2,
            F_req, Tb, PbP2, Pe, gamma, R);
505         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
506         valid(k) = true; status(k) = 0;
507     catch ME
508         valid(k) = false; status(k) = classify_error(ME.message
            );
509     end
510 end
511 end
512
513 function [eta_th, eta_p, eta_o, valid, status] = sweep_sp(
      sp_vec, P1, T1, M1, M2, F_req, Tb, PbP2, P4P1, gamma, R)
514 N = numel(sp_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
      NaN(1,N); valid = false(1,N); status = zeros(1,N);
515 for k = 1:N
516     try
517         Pe = P4P1 * P1;
518         [eth, ep] = single_point(P1, T1, M1, sp_vec(k), M2,
            F_req, Tb, PbP2, Pe, gamma, R);
519         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
520         valid(k) = true; status(k) = 0;
521     catch ME
522         valid(k) = false; status(k) = classify_error(ME.message
            );
523     end
524 end
525 end
526
527 function [eta_th, eta_p, eta_o, valid, status] = sweep_M2(
      M2_vec, P1, T1, M1, sp, F_req, Tb, PbP2, P4P1, gamma, R)
528 N = numel(M2_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
      NaN(1,N); valid = false(1,N); status = zeros(1,N);
529 for k = 1:N
530     try
531         Pe = P4P1 * P1;
532         [eth, ep] = single_point(P1, T1, M1, sp, M2_vec(k),
            F_req, Tb, PbP2, Pe, gamma, R);
533         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
534         valid(k) = true; status(k) = 0;

```

```

535     catch ME
536         valid(k) = false; status(k) = classify_error(ME.message
537             );
538     end
539 end
540
541 function [eta_th, eta_p, eta_o, valid, status] = sweep_Tb(
542     Tb_vec, P1, T1, M1, sp, M2, F_req, PbP2, P4P1, gamma, R)
543 N = numel(Tb_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
544     NaN(1,N); valid = false(1,N); status = zeros(1,N);
545 for k = 1:N
546     try
547         Pe = P4P1 * P1;
548         [eth, ep] = single_point(P1, T1, M1, sp, M2, F_req,
549             Tb_vec(k), PbP2, Pe, gamma, R);
550         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
551         valid(k) = true; status(k) = 0;
552     catch ME
553         valid(k) = false; status(k) = classify_error(ME.message
554             );
555     end
556 end
557 end
558
559 function [eta_th, eta_p, eta_o, valid, status] = sweep_PbP2(
560     PbP2_vec, P1, T1, M1, sp, M2, F_req, Tb, P4P1, gamma, R)
561 N = numel(PbP2_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
562     NaN(1,N); valid = false(1,N); status = zeros(1,N);
563 for k = 1:N
564     try
565         Pe = P4P1 * P1;
566         [eth, ep] = single_point(P1, T1, M1, sp, M2, F_req, Tb,
567             PbP2_vec(k), Pe, gamma, R);
568         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
569         valid(k) = true; status(k) = 0;
570     catch ME
571         valid(k) = false; status(k) = classify_error(ME.message
572             );
573     end
574 end
575 end
576
577 function [eta_th, eta_p, eta_o, valid, status] = sweep_F(F_vec,
578     P1, T1, M1, sp, M2, Tb, PbP2, P4P1, gamma, R)
579 N = numel(F_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o =
580     NaN(1,N); valid = false(1,N); status = zeros(1,N);
581 for k = 1:N

```



```

572     try
573         Pe = P4P1 * P1;
574         [eth, ep] = single_point(P1, T1, M1, sp, M2, F_vec(k),
575                                 Tb, PbP2, Pe, gamma, R);
576         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
577         valid(k) = true; status(k) = 0;
578     catch ME
579         valid(k) = false; status(k) = classify_error(ME.message
580             );
581     end
582 end
583 function [eta_th, eta_p, eta_o, valid, status] = sweep_P4P1(
584     P4P1_vec, P1, T1, M1, sp, M2, F_req, Tb, PbP2, gamma, R)
585 N = numel(P4P1_vec); eta_th = NaN(1,N); eta_p = NaN(1,N); eta_o
586     = NaN(1,N); valid = false(1,N); status = zeros(1,N);
587 for k = 1:N
588     try
589         Pe = P4P1_vec(k) * P1;
590         [eth, ep] = single_point(P1, T1, M1, sp, M2, F_req, Tb,
591                                 PbP2, Pe, gamma, R);
592         eta_th(k) = eth; eta_p(k) = ep; eta_o(k) = eth * ep;
593         valid(k) = true; status(k) = 0;
594     catch ME
595         valid(k) = false; status(k) = classify_error(ME.message
596             );
597     end
598 end
599 end
600 %% PLOTTING FUNCTIONS
601 function plot_efficiencies_choking(x, eta_th, eta_p, eta_o,
602     valid, xlabel, ttl)
603 figure('Color','white','Position',[100 100 780 500]);% Create a
604     new white figure window
605 hold on; % Keep all plotted objects on the same axes
606 invalid = ~valid;% Identify invalid solution points
607 if any(invalid) % If invalid points exist, shade them
608     segments = logical_segments(invalid);% Find contiguous
609         invalid regions
610     for i = 1:size(segments,1) % Loop over all invalid regions
611         idx1 = segments(i,1); % Start index of invalid
612             segment
613         idx2 = segments(i,2); % End index of invalid segment
614         x1 = x(idx1); % Start x-value of invalid
615             segment

```

```

608         x2 = x(idx2);           % End x-value of invalid
           segment
609         fill([x1 x2 x2 x1], [0 0 1 1], [1 0.80 0.80], ...
610             'EdgeColor','none', 'HandleVisibility','off'); %
           Shade invalid region in light red
611     end
612 end
613 plot(x(valid), eta_th(valid), '--', 'LineWidth', 2.0, '
        DisplayName', 'Thermodynamic \eta_{th}'); % Plot thermal
        efficiency
614 plot(x(valid), eta_p(valid), '--', 'LineWidth', 2.0, '
        DisplayName', 'Propulsive \eta_p'); % Plot propulsive
        efficiency
615 plot(x(valid), eta_o(valid), '-', 'LineWidth', 2.5, '
        DisplayName', 'Overall \eta_o'); % Plot overall
        efficiency
616 yline(0.61, ':', 'Color', 'r', 'LineWidth', 2.0, 'DisplayName',
        '61% Reference'); % Red dotted reference
        line at 61%
617 if any(invalid) % Add legend item for invalid region if needed
618     patch(nan, nan, [1 0.80 0.80], 'DisplayName', 'Invalid
        region'); % Dummy patch for legend
619 end
620 xlabel(xlbl, 'FontSize', 13, 'FontWeight', 'bold'); % Set x-
        axis label
621 ylabel('Efficiency', 'FontSize', 13, 'FontWeight', 'bold'); %
        Set y-axis label
622 title(ttl, 'FontSize', 14, 'FontWeight', 'bold'); % Set plot
        title
623 legend('Location','best','FontSize',11); % Show legend
624 grid on; box on; % Turn grid and box on
625 ylim([0 1]); % Set y-axis range to [0,1]
626 set(gca,'FontSize',11,'LineWidth',1.0); % Format axes
627 hold off; % Release current axes
628 end
629 function segments = logical_segments(mask)
630 d = diff([false, mask, false]);
631 % Detect start and end points of true segments
632 starts = find(d == 1); % Find segment start indices
633 ends = find(d == -1) - 1; % Find segment end indices
634 segments = [starts(:), ends(:)];
635 % Return start/end index pairs
636 end
637
638 %% STATUS CLASSIFICATION / PRINT
639 function code = classify_error(msg)
640 if contains(msg, 'Combustion failed') % Identify combustor
        failures

```

```

641     code = 1; % Assign code 1
642 elseif contains(msg, 'Nozzle failed') % Identify nozzle
    failures
643     code = 2; % Assign code 2
644 elseif contains(msg, 'Invalid efficiency quantities') %
    Identify invalid efficiency errors
645     code = 3; % Assign code 3
646 elseif contains(msg, 'Thrust sizing mismatch') % Identify
    thrust mismatch errors
647     code = 4; % Assign code 4
648 elseif contains(msg, 'Complex Mach') || contains(msg, 'Complex'
    )
649     code = 5; % Assign code 5
650 elseif contains(msg, 'Negative') || contains(msg, 'zero') ||
    contains(msg, 'invalid')
651     code = 6; % Assign code 6
652 else
653     code = 7; % Assign code 7 to all other numerical failures
654 end
655 end
656
657 function print_status_summary(name, status)
658 fprintf('\n%s:\n', name); % Print
    sweep name
659 fprintf(' feasible points = %d\n', sum(status==0));
660 % Count feasible points
661 fprintf(' combustion failed = %d\n', sum(status==1));
662 % Count combustor failures
663 fprintf(' nozzle failed = %d\n', sum(status==2));
664 % Count nozzle failures
665 fprintf(' invalid efficiency = %d\n', sum(status==3));
666 % Count invalid efficiency cases
667 fprintf(' thrust sizing mismatch = %d\n', sum(status==4));
668 % Count thrust mismatch cases
669 fprintf(' complex solution = %d\n', sum(status==5));
670 % Count complex-solution cases
671 fprintf(' negative/invalid area = %d\n', sum(status==6));
672 % Count negative or invalid-area cases
673 fprintf(' other numerical = %d\n', sum(status==7));
674 % Count other numerical failures
675 end
676
677 %% HELPER FUNCTIONS (used in various different module/
    component definition blocks)
678 function res = area_mach_func(M, gamma)
679 res = (1 ./ M) .* ((2 / (gamma + 1)) .* (1 + ((gamma - 1) / 2)
    .* M.^2)) ...
680     .^ ((gamma + 1) / (2 * (gamma - 1)));

```

```

681 % Standard isentropic area-Mach relation
682 end
683
684 function Ms = inverse_normal_shock(static_pressure_ratio, gamma
    )
685 Ms = ((static_pressure_ratio * (gamma + 1) + (gamma - 1)) / (2
    * gamma))^0.5;
686 % Invert normal-shock pressure relation
687 end
688
689 function p_ratio = norm_shock_press_stag(M, gamma)
690 p_ratio = ((gamma + 1) * M.^2 ./ (2 + (gamma - 1) * M.^2)).^(
    gamma / (gamma - 1)) .* ...
691     ((gamma + 1) ./ (2 * gamma * M.^2 - (gamma - 1))).^(1
    / (gamma - 1));
692 % Stagnation-pressure loss across shock
693 end
694
695 function p_ratio = isentropic_press_ratio(M, gamma)
696 p_ratio = (1 + ((gamma - 1) / 2) * M.^2) .^ (gamma / (gamma -
    1));
697 % Isentropic pressure ratio P0/P
698 end
699
700 function t_ratio = isentropic_temp_ratio(M, gamma)
701 t_ratio = 1 + ((gamma - 1) / 2) * M.^2;
702 % Isentropic temperature ratio T0/T
703 end
704
705 function f = obtainMb(Mb, left_mom, mdot2, Pb, Tb, gamma, R)
706 rhob = Pb / (R * Tb); % Density at station b
707 ab = sqrt(gamma * R * Tb); % Speed of sound at station b
708 ub = Mb * ab; % Velocity at station b
709 Ab = mdot2 / (rhob * ub); % Area at station b
710 right_mom = (rhob * ub^2 + Pb) * Ab; % Momentum-flux term at
    station b
711 f = left_mom - right_mom; % Residual of momentum equation
712 end
713
714 function Me = solve_exit_mach(PeP0, gamma)
715 f = @(M) (1 + (gamma - 1) / 2 .* M.^2).^(-gamma / (gamma - 1))
    - PeP0; % Exit-pressure matching function
716 Me = fzero(f, [1.01 10]); % Solve supersonic exit Mach number
717 end

```

3 Results

3.1 Verification and Validation

3.1.1 Inputs

The inputs are taken from the reference data provided in Lecture 3 (Birch, 2026) and entered into the source code. The value of normal shock strength is referred to P_y/P_x in the lecture material (Birch, 2026).

Table 2: Lists of Inputs Required

Required Inputs	Values of Inputs
Free-stream pressure	70000 Pa
Free-stream temperature	210 K
Flight Mach number	2.8
Normal shock strength (P_{C1}/P_1)	1.245
Burner entry Mach number	0.2
Burner temperature	1700 K
Burner pressure ratio (P_b/P_2)	1.0
Exhaust pressure ratio (P_4/P_1)	1.0
Required thrust	2000 N

3.1.2 Outputs

The reference data provided in Lecture 3 (Birch, 2026) is applied to verify the accuracy of the outputs produced by the source codes. The lecture material only provides the ratio of the inlet area to the respective station areas. The initial input of the inlet area for the reference result in Table 3 is assumed to be the same value with the output of the source codes.

Table 3: Lists of Results between Codes and Reference

Required Outputs	Code Outputs [m^2]	Reference Results [m^2]	Difference [%]
Inlet area A_1	0.003167		–
Inlet throat area A_{C1}	0.000905	0.000909	0.440
Burner entry area A_2	0.002685	0.002698	0.482
Burner exit area A_b	0.002270	0.002285	0.656
Nozzle throat area A_{C2}	0.001489	0.001495	0.401
Exhaust area A_4	0.005526	0.005514	-0.218

The comparison presented in Table 3 demonstrates that the code outputs are in close agreement with the reference results, thereby confirming the verification and validation of the source codes. Minor deviations are acceptable due to the assumptions adopted throughout the analysis.

3.2 Plots and Justification

To examine the influence of individual input parameters on the results, the One-Variable-At-a-Time (OVAT) method (Simmons, 2023) is employed, whereby only one variable is varied while all remaining parameters are held constant at the baseline values specified in Table 2. The resulting plots demonstrate the relationship between the engine efficiencies and each input variable, accompanied by critical discussion.

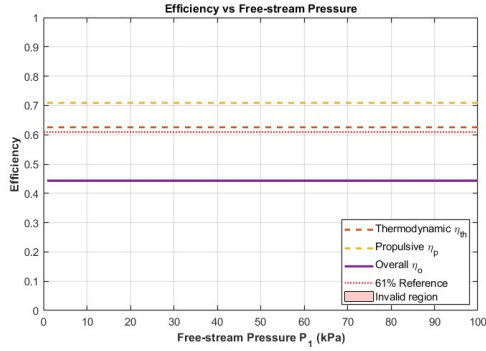


Figure 2: Efficiency vs. Free Stream Pressure

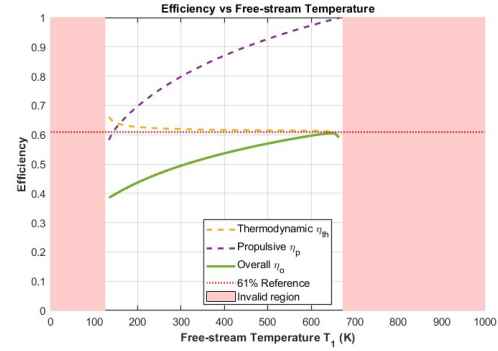


Figure 3: Efficiency vs. Free Stream Temperature

Justification of Figure 2: Scaling of the pressure values gradually, while keeping Mach values and temperature constant, produces no change in efficiencies since all three efficiencies are direct functions of Mach and temperature values only. Hence above plots are feasible.

Justification of Figure 3: With an increase in the free stream temperature, the speed of sound increases ($a_1 = \sqrt{\gamma RT}$), and inlet velocity increases for a constant Mach ($V_1 = M_1 a_1$). For a fixed value of T_b , T_b/T_1 falls, thereby making thermal efficiency fall gradually and slightly, because less relative heat addition is required. On the other hand propulsive efficiency grows more strongly with V_1 increase. Thus, overall efficiency rises noticeably sharply. Invalid regions depict unstart and engine melting regions accurately.

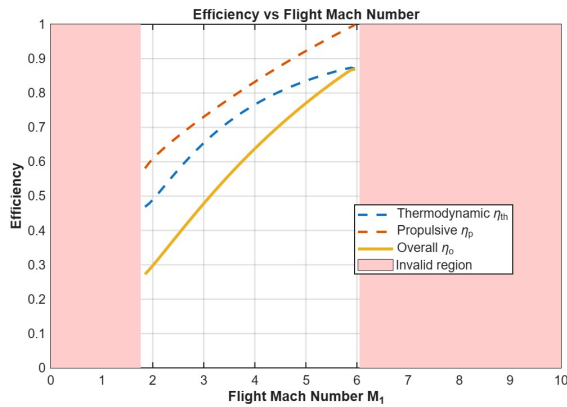


Figure 4: Efficiency vs. Flight Mach Number

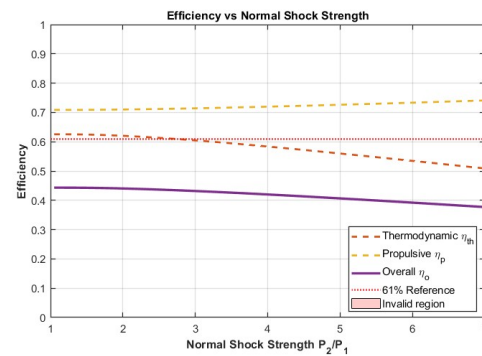


Figure 5: Efficiency vs. Normal Shock Strength

Justification of Figure 4: Higher M_1 value increases V_4 by increasing the P_{o1} and T_{o1} , thereby increasing propulsive efficiency as V_1 increases with a better velocity matching with V_4 . The thermal efficiency grows because the kinetic gain is sharper than the energy input in this case.

Justification of Figure 5: The three efficiencies are affected minimally by the shock strength here since the pressure loss brought by the shock affects P_o minimally, keeping it close to unity throughout the sweep values. Thus, there are minimal changes in P_{o2} , therefore very minute changes in the overall efficiency, although still in a decreasing fashion, thus being feasibly correct.

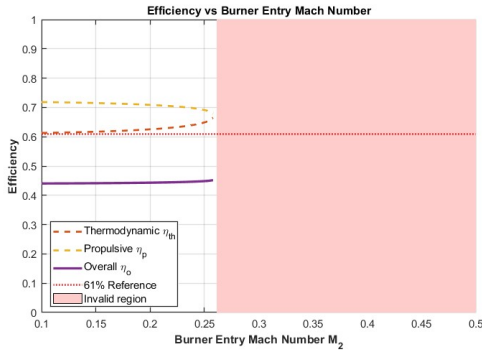


Figure 6: Efficiency vs. Burner entry mach number

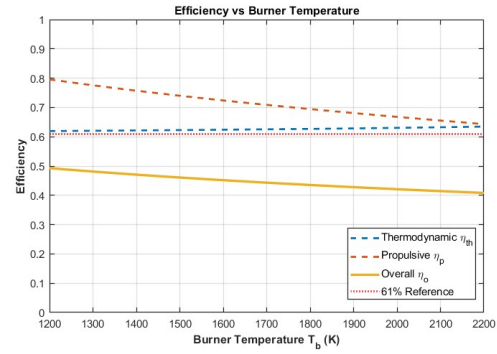


Figure 7: Efficiency vs. Burner temperature

Justification of Figure 6: Increasing the Mach number at the combustion chamber entry slightly enhances thermodynamic efficiency due to increased kinetic energy. However, as the difference between the exhaust jet velocity and the free-stream speed continues to grow, propulsive efficiency diminishes slightly. Therefore, the overall efficiency $\eta_o = \eta_{th}\eta_p$ remains relatively constant.

For larger values of M_2 , it cannot produce a physically valid subsonic solution, and the operating point is classified as an invalid region in the parametric sweep due to the following equation:

$$M_b = \frac{1}{2} \sqrt{\frac{T_2}{T_b} \left(M_2 + \frac{1}{\gamma M_2} \right)} \pm \frac{1}{2} \sqrt{\frac{T_2}{T_b} \left(M_2 + \frac{1}{\gamma M_2} \right)^2 - \frac{4}{\gamma}} \quad (22)$$

Justification of Figure 7: Increasing the burner exit temperature T_b slightly improves the thermodynamic efficiency by increasing the available thermal energy in the flow and raising the exhaust velocity after nozzle expansion. However, the propulsive efficiency decreases because the higher exhaust velocity increases the denominator of equation 8. Consequently, the reduction in propulsive efficiency dominates the small increase in thermodynamic efficiency, leading to a slight decrease in overall efficiency. This occurs because the reduction in propulsive efficiency is greater than the small increase in thermodynamic efficiency.

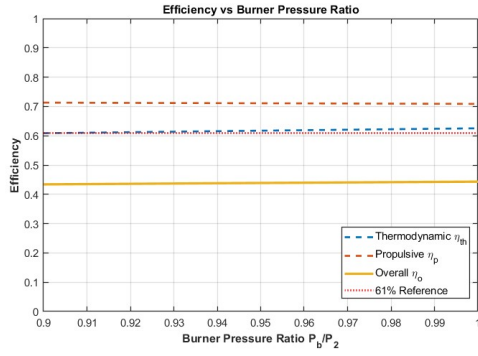


Figure 8: Efficiency vs. Burner Pressure Ratio

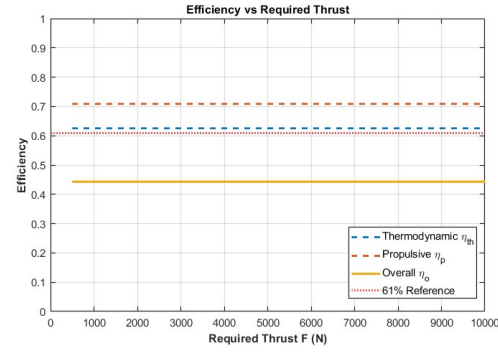


Figure 9: Efficiency vs. Required Thrust

Justification of Figure 8: As the burner pressure ratio P_b/P_2 increases from 0.5 to 1.0, pressure losses within the combustor decrease. This raises the stagnation pressure at the nozzle inlet, enhancing exhaust velocity and slightly improving thermodynamic efficiency. Although propulsive efficiency experiences a marginal reduction, the gain in thermodynamic efficiency outweighs this, ultimately resulting in a slight overall improvement in engine efficiency.

Justification of Figure 9: The exit velocity is determined by stagnation conditions and the exit mach number. Since the pressure ratio, velocity ratio and nozzle geometry remain fixed, the exit velocity and then specific thrust remains approximately constant as shown in Figure 9. Thermodynamics efficiency depends on the exit velocity and combustor exit temperature, while propulsive efficiency depends on the velocity ratio. Because these quantities remain unchanged in the presented analysis, overall efficiency remains nearly constant. This indicates that thrust can be increased by increasing the mass flow rate, rather than changing thermodynamic state of the flow.

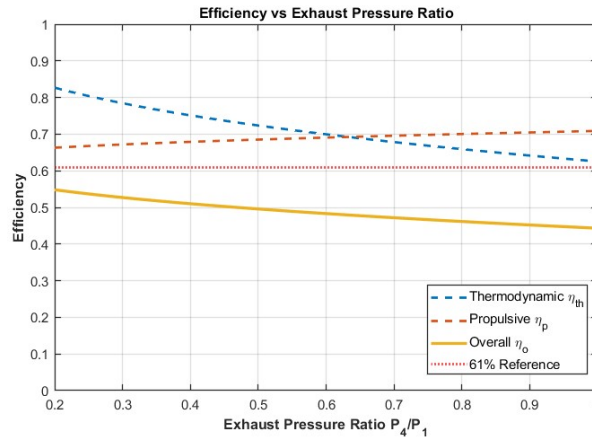


Figure 10: Efficiency vs. Exhaust Pressure Ratio

Justification of Figure 10: Also, as the exhaust pressure ratio increases, the degree of nozzle expansion decreases, reducing the exit mach number and jet velocity. The reduced jet velocity improves propulsive efficiency as the jet velocity approaches the freestream velocity. This contrasting effect gradually reduces overall efficiency, as shown in Figure 10.

4 Conclusion

The diffuser model predicts trends with physical consistencies, where free stream pressure has no effect on efficiencies and total pressure recovery falls with increasing normal shock strength. The code, however, overestimates the shock losses at higher Mach values from a real-case point of view, since it neglects the oblique shock pre-compression phase. The omission of shock-boundary layer interactions, boundary layer growth and viscous heating adds to the deviations seen in code results for total pressure losses, making the model less conservative than reality.

The combustor model produces physically reasonable results, indicating that the MATLAB code is working correctly. The efficiency diagram generated by variations in burner entry Mach number, burner temperature, and burner pressure ratio is same with the expected ramjet behaviors. The model also correctly plots an invalid region at large burner entry Mach numbers where no subsonic combustor solution exists.

The nozzle model is a conservative estimate of thrust-producing mass flow, as losses such as friction are neglected and the flow is assumed isentropic. In practical applications for all parts of the ramjet, more advanced models that account for non-isentropic effects and real fluid behaviour would be required to obtain more accurate performance and predictions.

4.1 Extended Critical Analysis:

- **Real-world extensions of the model (Engine unstart case analysis):** The **Kantrowitz limit** sets a more critical lower bound for the throat's cross-section below which the normal shock cannot be swallowed transiently. This gives a more conservative touch to the unstart boundary condition at the throat than the simple $M_5 = M_1$ used here in the diffuser module of the code for $M_5 < 1.3$. If the code were shifted to a more geometric contour computing template from its thermodynamic state-solving base, it would consider the contraction ratios to a better approximation considering the physical duct geometry, thus making it easier to analyse the engine unstart condition.
- **Weak shock—Optimal for overall efficiency:** If an oblique shock array is considered before the terminating normal shock in a gradual fashion where flow deviation is continuously discretized smoothly and at a constant rate in the form of a ramp, a much more realistically feasible normal shock can be obtained rather than just taking a single value of normal shock strength as input, not knowing where exactly it would be located and whether such values could exist for a given design of a ramjet.

References

- Kumar, N. and Yadav, V.A. (2022) 'Preliminary design of a ramjet engine: an analytical approach', *International Journal of Analytical Experimental and Finite Element Analysis (IJAEFEA)*, 9(3). doi:10.26706/jtfs.3.2.20220901. pages
- Wéber, R. and Luidens, R.W. (1956) 'Analysis of ram-jet engine performance including effects of component changes', *NASA Technical Reports Server (NASA)*. Available at: <https://ntrs.nasa.gov/api/citations/19930089332/downloads/19930089332.pdf>. pages
- Birch, D.M. (2026) 'Lecture 3 - $3 < M < 6$: Combustors and ramjet design', *AERO60003 Advanced Propulsion* [PowerPoint slides]. Department of Aeronautics, Imperial College London. pages
- Simmons, B.M. (2023) 'Evaluation of response surface experiment designs for distributed propulsion aircraft aero-propulsive modeling', *AIAA SCITECH 2023 Forum*. doi:10.2514/6.2023-2251. pages
- Ames Research Staff (1953) 'Equations, Tables and Charts for Compressible Flows', *National Advisory Committee for Aeronautics (NACA)*. Available at: <https://www.grc.nasa.gov/www/k-12/airplane/Images/naca1135.pdf> pages